

ULTRA-DETAILED BUILD DOCUMENTATION

Making of **MARKET DECODED**

The complete engineering record of a 60-second stock-market motion graphic built with Remotion 4 — from the environment audit and the numpy-synthesized 120 BPM soundtrack, through six hand-choreographed scenes, frame-by-frame quality assurance, to the final H.264 render. Every decision, every formula, nothing left out.

Remotion 4 · React 18 · TypeScript

6 scenes · 1,800 frames · 30 fps · 1920 × 1080

DURATION

60 seconds

SCENES

6 acts

DELIVERABLE

H.264 + AAC

BUILD TIME

5m 10s render

SEPTEMBER 2026 · Z.AI BUILD SERIES

Table of Contents

1. Project Overview and the Deliverable	4
1.1 The Brief	4
1.2 The Deliverable	4
1.3 The Creative Concept	5
2. Environment Audit and Project Setup	6
2.1 Auditing the Machine Before Writing Code	6
2.2 Scaffolding the Project	6
2.3 remotion.config.ts — Tuning the Renderer	7
2.4 TypeScript Configuration	8
3. Designing the 60 Seconds: Beat Grid and Storyboard	8
3.1 The Musical Clock Comes First	8
3.2 The Scene Map	9
3.3 Narrative Architecture: Mirrored and Resolved	9
3.4 Frame Budgeting Inside a Scene	10
4. The Soundtrack: A NumPy-Synthesized Score	10
4.1 Why the Music Is Code	10
4.2 The Signal Chain	10
4.3 Instrument Recipes	11
4.4 The Arrangement, Bar by Bar	11
4.5 The Mix Bus: Pump, Saturation, Loudness	12
5. Project Architecture: The Core Engine Files	13
5.1 Entry Point and the Composition	13
5.2 The Timeline Host: StockMarketVideo.tsx	13
5.3 theme.ts — One Source of Truth	14
5.4 lib.ts — The Animation Math Toolbox	15
6. The Reusable Component Toolkit	16
6.1 stage.tsx — The Stage Dressing	16
6.2 ui.tsx — The Typographic Actors	16
6.3 fx.tsx — The Impact Toolkit	17

6.4 charts.tsx — The Data Visualization Actors	17
6.5 procon.tsx — The Pros and Cons Card	18
7. Scene 1 — Intro: Glitch Title and Candle Skyline	18
7.1 Purpose and First Impressions	18
7.2 The Candle Skyline	18
7.3 Title, Clock Rings, and the Beat Drop	18
8. Scene 2 — The Hook: Self-Drawing Charts and a Live Ticker	19
8.1 The Two Questions	19
8.2 A Market That Draws Itself	19
8.3 Planting the Rivalry	20
9. Scene 3 — Fundamental Analysis (School 01)	20
9.1 Page 1: The Company Report Card	20
9.2 The Page Turn	21
10. Scene 4 — Technical Analysis (School 02)	21
10.1 Page 1: Anatomy of a Chart	21
10.2 The Bouncing Price Ball	22
10.3 RSI and Volume Panels	22
11. Scene 5 — Head-to-Head: The Versus Scene	23
11.1 A Stage Split in Two	23
11.2 The VS Emblem and the Four Rounds	23
11.3 The Verdict	23
12. Scene 6 — Outro: The Synthesis and the End Card	24
12.1 Merging the Two Schools	24
12.2 The Lesson, in Three Lines	24
12.3 The End Card and the Long Fade	24
13. Quality Assurance: The Frame Review Loop	25
13.1 Why Still Frames Are Enough	25
13.2 Six Bugs Found and Fixed	25
13.3 The Second Pass	26
14. Rendering and Delivery	26

14.1 Benchmark Before the Big Render	26
14.2 The Render Command, Dissected	27
14.3 Delivery	27
15. Complete File Inventory and Reproduction Guide	27
15.1 Every File and Its Role	27
15.2 Reproducing the Video from Scratch	28
15.3 Where to Take It Next	28
15.4 Performance Notes for Faster Renders	29

1. Project Overview and the Deliverable

This document is the complete, nothing-left-out engineering record of **Market Decoded**, a 60-second stock-market motion graphic that was designed and rendered entirely with **Remotion 4**, the React-based programmatic video framework. It explains every step of the build in chronological order: how the environment was audited, how the project was scaffolded, why the soundtrack was synthesized in NumPy before a single frame was animated, how the six scenes and twenty reusable components were written in TypeScript, how every animation formula works, and how fourteen still frames were inspected to catch six layout bugs before the final render. The goal of this document is simple: someone who has never opened the project should be able to understand, reproduce, and extend every part of it.

60 s
exact duration

1,800
frames

30 fps
frame rate

17.6 MB
final file size

1.1 The Brief

The original request had four hard requirements, and every one of them shaped an architectural decision downstream. First, the video had to be built with **Remotion**, downloaded and installed from scratch, which dictated a React 18 + TypeScript codebase rendered frame-by-frame by Chromium. Second, the motion graphics had to be based on the **stock market**, which led to a synthwave trading-terminal visual language: candlestick charts, ticker tapes, price counters, support and resistance levels, and an RSI oscillator. Third, the video had to **impress** with many mind-blowing effects, which motivated a custom effects toolkit (RGB-split glitch text, shockwave rings, spark bursts, light streaks, particles, scanlines) plus beat-synced animation. Fourth, it had to **teach** fundamental versus technical analysis including pros and cons of each, which produced the mirrored two-page scene structure and the head-to-head comparison scene.

- **Remotion, from scratch** — a clean npm project was created manually rather than from a starter template, keeping dependencies minimal and fully understood.
- **Stock-market theme** — every scene is built from real trading concepts: OHLC candles, moving averages, golden crosses, volume, revenue reports, and valuation gaps.
- **Spectacle** — every scene cut lands on a musical impact with a flash, shockwave, or spark burst, and ambient motion (particles, starfields, grid floors, ticker tapes) never stops.
- **Education** — both schools of analysis get an identical definition page and an identical pros-and-cons page, so the video stays fair and structured.

1.2 The Deliverable

The finished video is a single self-contained file with music muxed in. The exact specifications are listed below; every one of them is a consequence of a configuration choice explained later in this document.

Property	Value
File name	stock-market-decoded.mp4
Resolution	1920 × 1080 (Full HD, 16:9)
Frame rate	30 fps, exactly 1,800 frames
Duration	60.0 seconds
Video codec	H.264 (MP4 container)
Audio codec	AAC, 44.1 kHz stereo, synthesized in-house
File size	17.6 MB
Composition ID	StockMarketDecoded
Framework	Remotion 4.0 + React 18.3 + TypeScript 5.6

Table 1: Final video specification

1.3 The Creative Concept

The video is styled as a **retro-futuristic trading terminal**: a near-black navy sky, neon green and cyan data, gold accents for moments of insight, a perspective grid floor scrolling toward the horizon, and a persistent scanline-plus-vignette overlay that makes every frame feel like it is being displayed on an old CRT monitor. Orbitron (a squared, sci-fi display face) carries the headlines, Space Grotesk handles body copy, and JetBrains Mono renders every number and ticker — the monospaced digits guarantee that animated counters never jitter in width.

Color is used as a language throughout. Each school of analysis owns a hue, and each act of the story owns an accent, so the viewer learns the palette intuitively before the comparison scene even begins. The palette is defined once in a central theme file and referenced by every scene and component.

Hue	Hex	Meaning in the story
Neon green	#00ff9d	Fundamental analysis, bullish candles, positive values
Cyan	#22d3ff	Technical analysis, the chart itself, momentum
Gold	#ffd166	Insight and synthesis: the VS emblem, stamps, the merged arrow
Red / pink	#ff3b6b	Bearish candles, cons, resistance, danger
Violet	#a78bfa	The outro, where both schools merge
Deep navy	#04070f	The void behind everything — the terminal at night

Table 2: The color language of the video

2. Environment Audit and Project Setup

2.1 Auditing the Machine Before Writing Code

Before installing anything, the machine itself was audited, because Remotion renders video by screenshotting a headless Chromium thousands of times and encoding the result — the hardware determines both the render strategy and the expected wall-clock time. The audit found **Node v24** (well above Remotion's minimum), **2 CPU cores and 4 GB of RAM** (modest, which meant concurrency had to be capped), a working **ffmpeg** (Remotion bundles its own encoder, but a system ffmpeg is useful for sanity checks), and **NumPy** (which made the plan to synthesize the soundtrack in Python viable). Each finding fed directly into a decision recorded later in the configuration files.

Check	Result	Decision it drove
Node.js	v24.21	Remotion 4 CLI runs natively; no version shims needed
CPU cores	2	remotion.config.ts concurrency clamped to $\max(2, \min(6, \text{cores})) = 2$
Memory	4 GB	JPEG frame format chosen over PNG to keep memory pressure low
ffmpeg	present	Final H.264 + AAC muxing verified as available
NumPy	present	Soundtrack generated programmatically (Chapter 4)

Table 3: Environment audit and the decision each finding drove

2.2 Scaffolding the Project

The project lives in its own directory with a hand-written package.json. Choosing a manual scaffold over the official **create-video** template was deliberate: the template ships with demo comps, assets, and boilerplate that would all have to be deleted, whereas a minimal manifest produces a dependency tree where every package has a known reason to exist. Installation was a single npm install, and the resulting npm-install.log confirmed a clean resolution of all seven runtime packages.

```
{
  "name": "stock-market-decoded",
  "version": "1.0.0",
  "private": true,
  "description": "60-second stock market motion graphic -
    Fundamental vs Technical analysis, built with Remotion",
  "scripts": {
    "render": "remotion render src/index.ts StockMarketDecoded
      out/stock-market-decoded.mp4"
  },
  "dependencies": {
    "@fontsource/jetbrains-mono": "^5.0.0",
    "@fontsource/orbitron": "^5.0.0",
    "@fontsource/space-grotesk": "^5.0.0",
    "@remotion/cli": "^4.0.0",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "remotion": "^4.0.0"
  },
  "devDependencies": {
    "@types/react": "^18.3.12",
    "typescript": "^5.6.3"
  }
}
```

Listing 1: package.json — the entire dependency manifest

Three of these choices deserve justification. **@fontsource packages** ship the actual font files as npm modules and are imported as CSS inside the video component — the fonts are bundled locally, so frame 400 looks identical on a machine with no internet access, and Google Fonts can never fail mid-render or change metrics between renders. **Pinning Remotion and the CLI to the same major version** avoids subtle mismatches between the renderer and the runtime. **TypeScript in devDependencies only** is possible because Remotion bundles with esbuild and never runs tsc during rendering; the compiler exists purely for editor feedback and type safety, configured with noEmit.

2.3 remotion.config.ts — Tuning the Renderer

```
import os from 'os';
import {Config} from '@remotion/cli/config';

Config.setVideoImageFormat('jpeg');
Config.setOverwriteOutput(true);
Config.setConcurrency(Math.max(2, Math.min(6, os.cpus().length)));
Config.setChromiumOpenGLRenderer('angle');
```

Listing 2: remotion.config.ts, annotated line by line

Each line is a deliberate trade-off. **VideoImageFormat jpeg** trades a little compression quality for significantly faster frame encoding and lower memory use — at 1080p with heavy motion, JPEG artifacts are invisible, and the QA loop in Chapter 13 renders hundreds of stills faster. **OverwriteOutput true** makes the render command idempotent during iteration, so re-rendering after a fix never fails on an existing file. **Concurrency** is the number of browser tabs that render frames in parallel; the clamp formula resolved to 2 on

this 2-core machine, which prevents the 4 GB of RAM from being exhausted by four simultaneous Chromium instances. **ANGLE as the OpenGL renderer** selects the GPU-backed path for compositing CSS filters and box-shadows, which this video leans on heavily for its neon glow aesthetic — it produces more accurate blur and shadow output than the software SwiftShader fallback.

2.4 TypeScript Configuration

```
{
  "compilerOptions": {
    "target": "ES2018",
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "jsx": "react-jsx",
    "strict": true,
    "skipLibCheck": true,
    "esModuleInterop": true,
    "resolveJsonModule": true,
    "noEmit": true,
    "lib": ["DOM", "ES2018"]
  },
  "include": ["src", "remotion.config.ts"]
}
```

Listing 3: tsconfig.json

The important flags are **jsx: react-jsx** (the modern transform — no React import needed in every file), **strict: true** (every animation helper is typed, which caught several unit mistakes — frames versus seconds — before they could reach a render), and **noEmit** (Remotion's bundler is the only consumer of the source; tsc is a type-checker, not a build step). The DOM lib is required because Remotion components are ordinary React DOM trees styled with inline CSS.

3. Designing the 60 Seconds: Beat Grid and Storyboard

3.1 The Musical Clock Comes First

The single most important design decision was made before any storyboard: **the video is a musical piece, and the animation grid locks to it**. The soundtrack runs at 120 BPM, which produces beautifully round numbers at 30 fps: one beat is 0.5 seconds = 15 frames, one bar of 4/4 is 2 seconds = 60 frames, and the full 60 seconds is exactly 30 bars. Because the beat grid divides the timeline so cleanly, every scene boundary can sit on a bar line, every pulse animation can fire on a kick drum, and every impact effect can coincide with the musical hit that the soundtrack generator places at the same frame. This is why the video feels edited rather than merely animated.

120 BPM

tempo

0.5 s

one beat

15 f

one beat in frames

30 bars

full track

3.2 The Scene Map

Six scenes partition the 1,800 frames. The map is stored as a single SCENES object in the theme file and is imported by every scene and by the progress HUD, so the timing system has exactly one source of truth — changing one number re-times the entire video, including the HUD chapter markers and the music (which was generated from the same map).

Scene	Name	Frames	Seconds	Job in the story
S1	Intro	0 - 119	0:00 - 0:04	Brand the video; establish the synthwave terminal
S2	Hook	120 - 299	0:04 - 0:10	Ask the two questions; let a chart draw itself
S3	Fundamental	300 - 659	0:10 - 0:22	Teach school 01; its pros and cons
S4	Technical	660 - 1019	0:22 - 0:34	Teach school 02; its pros and cons
S5	Versus	1020 - 1379	0:34 - 0:46	Head-to-head comparison; the synthesis stamp
S6	Outro	1380 - 1799	0:46 - 1:00	Merge both schools; end card; fade to black

Table 4: The scene map — boundaries locked to 2-second musical bars

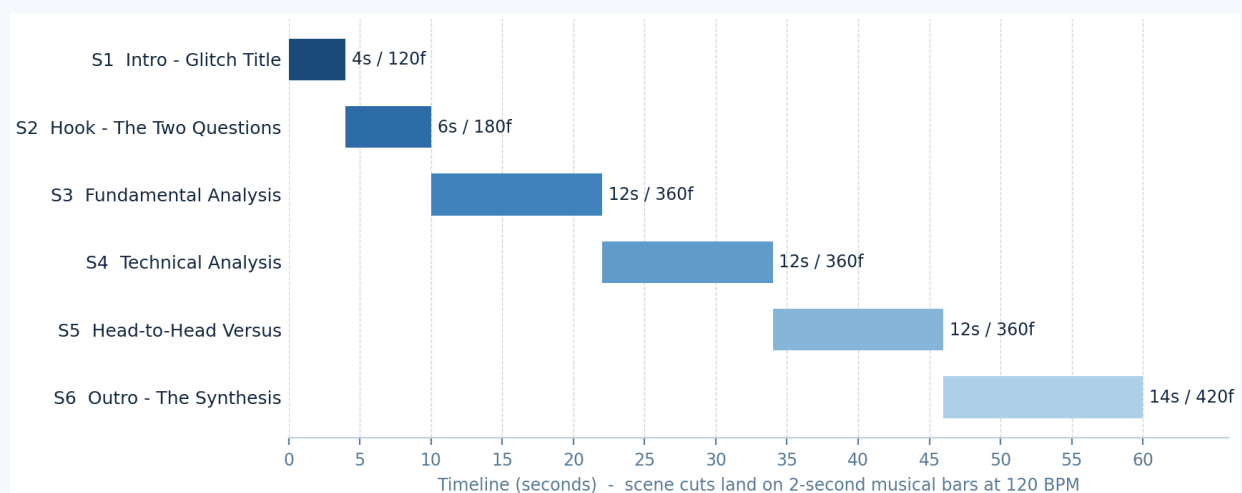


Figure 1: The six scenes across the 60-second timeline; every cut lands on a musical bar

3.3 Narrative Architecture: Mirrored and Resolved

The story follows a classic tease - teach - compare - resolve arc. Scenes 1 and 2 tease by dramatizing the investor's two questions (**what** to buy, and **when** to buy it), which is precisely the dichotomy between the two schools. Scenes 3 and 4 teach, and they are deliberately **mirror images** of each other: both open with a badge-and-title header, both devote their first page to a living diagram of the method (a company report card

versus an annotated price chart), both flip to an identical pros-and-cons page, and both close with a famous one-line quotation. Mirroring is a fairness device — the video never visually favors either school — and it halved the design work because one layout serves both scenes.

Scene 5 compares the schools in four head-to-head rows and resolves the tension with a stamped verdict — **WHY NOT BOTH?** — and Scene 6 completes the arc by literally animating the two schools' charts flying toward each other, merging into a single rising golden arrow that carries the end card. The final line, **SMART MONEY USES BOTH**, is the thesis of the entire video.

3.4 Frame Budgeting Inside a Scene

Each scene then budgets its own 30 or 60 frames-per-second-of-story against the beat grid. Scene 3, for example, spends its 360 frames as follows: frames 0-30 header entrance, 30-60 report card spring-in, 40-104 the three revenue counters rolling, 98 the UNDERVALUED stamp slam, 126 the caption line, and 156-172 a page transition (the whole page fades and slides up 34 pixels), leaving 176-360 for the pros-and-cons page and the closing quotation at frame 300. The same discipline applies to every scene, and the per-element timings are documented scene by scene in Chapters 7 through 12.

4. The Soundtrack: A NumPy-Synthesized Score

4.1 Why the Music Is Code

The soundtrack was written as a Python script (`gen_music.py`) that renders a 60-second stereo WAV with NumPy and writes it straight into the Remotion public folder. Three reasons justified building music from raw samples instead of sourcing a track. **Licensing and size:** a generated WAV has zero licensing constraints and no external asset to lose. **Exact synchronization:** the script imports the same scene map the video uses, so risers and impacts are placed at 4, 10, 22, 34, and 46 seconds — the exact frames where the scene cuts happen, to the sample. **Total control:** energy, instrumentation, and arrangement can be tuned per section and re-rendered in seconds; the script even prints a per-5-second RMS table so the loudness arc can be verified numerically.

4.2 The Signal Chain

Audio is rendered at 44,100 Hz into three stereo buses, summed at the end: a **musical bus** (pads, bass, arpeggios — the bus that gets sidechain-pumped), a **drum bus** (kick, hats, claps — never pumped, so the groove stays punchy), and an **FX bus** (risers and impacts on scene cuts). A helper called `add()` schedules any signal at any time with gain and constant-power panning (the pan value drives a cosine/sine pair), while two FFT helpers — lowpass and highpass — implement 2nd-order filters directly on the spectrum via `rfft`, avoiding any external DSP library.

```
def add(bus, t0, sig, gain=1.0, pan=0.0):
    i0 = int(round(t0 * SR))          # seconds -> sample index
    gl = gain * np.cos((pan + 1) * np.pi / 4)  # constant-power pan
    gr = gain * np.sin((pan + 1) * np.pi / 4)
    bus[0][i0:i0+len(sig)] += sig * gl  # left channel
    bus[1][i0:i0+len(sig)] += sig * gr  # right channel

def lowpass(x, fc, order=2):
    X = np.fft.rfft(x)
    f = np.fft.rfftfreq(len(x), 1.0 / SR)
    X = X / np.sqrt(1.0 + (f / max(fc, 1.0)) ** (2 * order))
    return np.fft.irfft(X, len(x))
```

Listing 4: Bus scheduling and spectral filters (abridged from gen_music.py)

4.3 Instrument Recipes

Every drum and synth voice is a small synthesis recipe built from oscillators, noise, envelopes, and the two filters above. The table below records the exact recipe of each voice, because these numbers are the difference between a generic beep and a record that sounds intentionally produced.

Voice	Recipe
Kick	Sine with pitch sweep $165 \cdot e^{(-17t)} + 44$ Hz, exponential decay (rate 8.5), plus a high-passed (1600 Hz) noise click at 0.5 gain for the beater
Hat	High-passed (7200 Hz) white noise; decay rate 95 closed, 34 open
Clap	Three band-limited noise bursts at 0, 13, and 27 ms over an $e^{(-21t)}$ envelope — the classic triple-flam clap
Saw stack	3 detuned sawtooths (sign of sine, detune $\pm 0.7\%$), the basis of pads and bass
Bass	2-oscillator detuned saw plus a sine sub, low-passed at 330 Hz, 4 ms attack
Pad	Saw stack + $0.35 \times$ sine bed, low-passed per section (520-1400 Hz), 0.5 s attack and 0.7 s release
Pluck	Sum of sine harmonics $1.0 / 0.4 / 0.15$ with $e^{(-16t)}$ decay — the arpeggio voice
Riser	High-passed noise shaped $t^{2.4}$ + a 200-1100 Hz sine sweep, tremolo at 8 Hz
Impact	Sub sine sweeping $78 \rightarrow 38$ Hz with $e^{(-3t)}$ + low-passed (520 Hz) noise boom
Shimmer	Bare sine at 880 / 1318.5 Hz with slow decay, panned wide, for the final chord

Table 5: Instrument synthesis recipes

4.4 The Arrangement, Bar by Bar

The piece is 30 bars of 4/4 in A minor, cycling the classic Am - F - C - G progression (voiced at 110 / 87.3 / 130.8 / 98 Hz roots). The arrangement arc mirrors the video: two sparse bars under the intro (pads and tick hats only), the full groove entering with the hook at bar 2, the arpeggio joining at bar 5 for the teaching sections, a brighter open filter during the versus scene, and a fading outro where the drums decay bar by bar until a final

resolved Am chord with a sub bass and two shimmer notes ends the piece at $t = 56$ s.

Bars	Section	What plays
0-1	Intro (S1)	Pads + tick hats; no drums — space for the title
2-4	Hook (S2)	Kick 4-on-floor, offbeat hats, claps on 2 and 4, 8th-note bass
5-10	Fundamental (S3)	16th-note ping-pong arpeggio joins; pads at 0.46 gain
11-16	Technical (S4)	Same engine, chords F-F-G-G-Am-Am; arpeggio softer
17-22	Versus (S5)	Brightest pads (cutoff 1150 Hz), loudest arpeggio
23-29	Outro (S6)	Drums fade $0.9 \rightarrow 0.32$, sustained bass, final Am chord + shimmer

Table 6: Arrangement map — bars, sections, energy

4.5 The Mix Bus: Pump, Saturation, Loudness

Two finishing moves give the track its modern, glued sound. First, **sidechain pumping**: after $t = 4$ s the musical bus is multiplied by a decaying curve on every beat, so the pads and bass audibly duck out of the kick's way — the same trick used in dance music, expressed here as one line of NumPy. Second, **tanh saturation** plus peak normalization: the mix is soft-clipped with $\tanh(x \cdot 1.25) \cdot 0.85$ to add harmonic density, then normalized to a 0.95 peak, with a 50 ms fade-in and a fade-out starting at 59.7 s so the file ends cleanly on the video's last frame.

```
tt = np.arange(N) / SR
pump = np.ones(N)
m = tt > 4.0
pump[m] = 1.0 - 0.36 * np.exp(-((tt[m] % BEAT) / 0.09))

L = MUS_L[:N] * pump + DRM_L[:N] + FX_L[:N]
R = MUS_R[:N] * pump + DRM_R[:N] + FX_R[:N]

L *= np.clip(tt / 0.05, 0, 1) * np.clip((59.7 - tt) / 0.3, 0, 1)
mix = np.tanh(np.stack([L, R], axis=1) * 1.25) * 0.85
mix = mix / np.max(np.abs(mix)) * 0.95 # final peak normalize
```

Listing 5: The sidechain pump and master chain

The script ends with verification: it prints the detected peak and an RMS loudness reading for each 5-second block of the track. That printout confirmed the intended dynamics — quiet intro (RMS 0.06), full energy through the teaching scenes (0.11-0.13), a peak in the versus scene, and a controlled decay into the outro — without ever opening an audio editor.

5. Project Architecture: The Core Engine Files

Remotion makes no demands on how a project is organized, so the architecture was designed around one principle: **timing, color, and math each get exactly one home**. Seven TypeScript files implement the entire video — an entry point, a composition declaration, a timeline host, a design-system pair, and the six scenes on top of five component libraries. The full tree is inventoried in Chapter 15; this chapter walks through the four engine files that everything else imports.

5.1 Entry Point and the Composition

Two tiny files boot the whole system. `src/index.ts` calls `registerRoot`, and `src/Root.tsx` declares the composition — the video's top-level contract: its ID (used by every render and studio command), its canvas, its frame rate, and its total length.

```
// index.ts
import {registerRoot} from 'remotion';
import {RemotionRoot} from './Root';
registerRoot(RemotionRoot);

// Root.tsx
import {Composition} from 'remotion';
import {StockMarketVideo} from './StockMarketVideo';

export const RemotionRoot: React.FC = () => {
  return (
    <Composition
      id="StockMarketDecoded"
      component={StockMarketVideo}
      durationInFrames={1800}
      fps={30}
      width={1920}
      height={1080}
      backgroundColor="#04070f"
    />
  );
};
```

Listing 10: `src/index.ts` and `src/Root.tsx` — the complete boot layer

The numbers here are the decisions of Chapter 3 expressed as code: 1800 frames at 30 fps is exactly 60 seconds, and 1920×1080 is the Full-HD canvas every layout coordinate in later chapters assumes. The background color `#04070f` is the deepest navy of the palette — it shows through anywhere no component paints, which after the QA pass is nowhere.

5.2 The Timeline Host: `StockMarketVideo.tsx`

```

<AbsoluteFill style={{backgroundColor: COLORS.bg, fontFamily: FONTS.body}}>
  <Audio src={staticFile('music.wav')} />

  <Sequence from={SCENES.intro.from} durationInFrames={SCENES.intro.dur}>
    <S1Intro />
  </Sequence>
  <Sequence from={SCENES.hook.from} durationInFrames={SCENES.hook.dur}>
    <S2Hook />
  </Sequence>
  {/* ... S3, S4, S5 follow the same pattern ... */}
  <Sequence from={SCENES.outro.from} durationInFrames={SCENES.outro.dur}>
    <S6Outro />
  </Sequence>

  <ProgressHUD />
  <ScreenFX />
</AbsoluteFill>

```

Listing 11: The timeline host (abridged)

Three architectural facts live in this file. First, the **Audio** tag mounts the soundtrack once for the whole video via `staticFile` (Remotion's resolver for the public folder), so audio stays in lockstep with frames no matter how the scenes change. Second, each scene is wrapped in a **Sequence**, which does two critical things: it mounts the scene only during its frame window (performance — six scenes never render simultaneously), and it **rewrites `useCurrentFrame()` for its children to be scene-local**. That is why every scene in Chapters 7-12 speaks in frames 0-N rather than global frames, and why the `SCENES` map in `theme.ts` can re-time the video without touching a single scene file. Third, the **ProgressHUD** and **ScreenFX** overlays sit after the sequences in JSX order, so they stack above every scene — chrome on top of content.

5.3 theme.ts — One Source of Truth

```

export const COLORS = {
  bg: '#04070f', panel: 'rgba(8, 14, 26, 0.78)',
  green: '#00ff9d', red: '#ff3b6b', cyan: '#22d3ff',
  gold: '#ffd166', violet: '#a78bfa', white: '#f4f7ff',
  muted: 'rgba(226, 236, 255, 0.6)',
  faint: 'rgba(226, 236, 255, 0.28)',
};

export const FONTS = {
  display: 'Orbitron, sans-serif',
  body: '"Space Grotesk", sans-serif',
  mono: '"JetBrains Mono", monospace',
};

// Global timeline (30 fps): scene boundaries on musical bars
export const SCENES = {
  intro: {from: 0, dur: 120},
  hook: {from: 120, dur: 180},
  fundamental: {from: 300, dur: 360},
  technical: {from: 660, dur: 360},
  versus: {from: 1020, dur: 360},
  outro: {from: 1380, dur: 420},
};

```

Listing 12: The design system in 28 lines

The ten color tokens and three font stacks of Chapter 1 are declared here once; no scene file contains a raw hex value for green or cyan (the few decorative whites excepted). Because the panel color is expressed as rgba, cards composite over the backdrop gradients rather than sitting as flat rectangles. The SCENES map doubles as documentation — the comment pins the invariant that boundaries land on musical bars, which the music generator (Chapter 4) independently encodes.

5.4 lib.ts — The Animation Math Toolbox

```
// Kick drum lands every 0.5 s => every 15 frames. Decaying pulse:
export const beat = (frame: number, strength = 1): number =>
  Math.exp(-(frame % 15) / 4) * strength;

// Catmull-Rom spline through points -> smooth SVG path string
export const smoothPath = (pts: Pt[]): string => {
  if (pts.length < 2) return '';
  let d = `M ${pts[0][0]} ${pts[0][1]}`;
  for (let i = 0; i < pts.length - 1; i++) {
    const p0 = pts[Math.max(0, i - 1)];
    const p1 = pts[i];
    const p2 = pts[i + 1];
    const p3 = pts[Math.min(pts.length - 1, i + 2)];
    const cx = p1[0] + (p2[0] - p0[0]) / 6; // tangent handles
    const cy = p1[1] + (p2[1] - p0[1]) / 6; // at 1/6 of the
    const c2x = p2[0] - (p3[0] - p1[0]) / 6; // neighbor delta
    const c2y = p2[1] - (p3[1] - p1[1]) / 6;
    d += ` C ${cx} ${cy}, ${c2x} ${c2y}, ${p2[0]} ${p2[1]}`;
  }
  return d;
};
```

Listing 13: The four helpers every scene leans on

beat() is the smallest function in the project and the reason the video feels alive: the modulo-15 pulse fires on every kick drum and decays with a time constant of 4 frames, giving a sharp attack and a 0.2-second tail. The VS emblem's pumping scale (Chapter 11) is one call to it. **smoothPath()** converts a polyline into a Catmull-Rom spline expressed as cubic Béziers: each segment's control points are placed one-sixth of the way along the neighboring points' delta, which yields a curve that passes through every data point with continuous tangents — the difference between the hook's silky price line and an angular connect-the-dots.

The remaining three exports are quieter but just load-bearing. **pointAt(pts, t)** walks a polyline by cumulative segment length, returning the exact point at fraction *t* of total arc length — this is how the self-drawing line's head dot rides the curve at the true drawing position instead of drifting off it on steep segments. **rnd(seed)** wraps Remotion's deterministic random: same seed string, same value, every render and every pass — all particles, stars, zig-zag vertices, and volume bars depend on it for flicker-free randomness. **clamp** is the extrapolation guard passed into every **interpolate()** call, so no animation overshoots its keyframes outside the defined range.

6. The Reusable Component Toolkit

Before any scene was written, five component files were built so that every scene composes small, tested, animated parts instead of raw divs. This is the highest-leverage decision in the whole project: the scenes in Chapters 7-12 read like stage directions because the toolkit hides the animation math. All timing inside components is driven by `useCurrentFrame()` — never by timers or CSS animations — which is what makes the output deterministic: frame 613 of a render is pixel-identical no matter how fast the machine is.

6.1 stage.tsx — The Stage Dressing

Stage components are the ambient environment shared across scenes. **Backdrop** stacks four gradients (a large accent-tinted radial from the top, a subtle floor bounce radial, a blue mid-field radial, and a vertical base gradient from `#05080f` to `#03050b`) plus an optional extra layer used by the intro for a gold halo. **StarField** generates *N* stars from a string seed — each star's *x*, *y*, radius, and phase come from Remotion's deterministic random, and opacity follows $0.1 + 0.35 \cdot (0.5 + 0.5 \cdot \sin(f \cdot 0.06 + \text{phase}))$, producing a slow, flicker-free twinkle. **GridFloor** is pure CSS 3D: a container with perspective 480 px holds a plane rotated `rotateX(64deg)`, textured with a 76 px grid drawn by two linear-gradients, scrolled by feeding `backgroundPosition` the frame number ($f \cdot \text{speed px}$), and faded at the horizon with a mask-image gradient — the retro synthwave floor.

TickerTape implements a seamless infinite marquee with two copies of a 12-symbol set (fictional tickers ZNIA, VLTA, QNTA, HELX, ORBT, KRYX, NVDX, AMBR, FLUX, TSMB, GLOW, PTQX — 320 px per item). The offset is $(f \cdot 3) \% \text{setWidth}$, the second copy trails exactly one set-width behind, and direction ± 1 flips it, which is why the intro can run two tapes in opposite directions without ever exposing a seam. Each quote carries a price and a percentage chip with an inline-SVG triangle. **ScreenFX** overlays the CRT feel globally: scanlines via a repeating-linear-gradient (1 px lines of 2.4% white every 4 px) and a vignette via a radial gradient that darkens the outer 42% of the frame. **ProgressHUD** is a player-style chrome: the active chapter name (derived from the SCENES map), an m:ss clock, a 6 px progress bar in a green → cyan → gold gradient, and white tick marks at every scene boundary; it fades in at frames 128-152 and out at 1758-1786 so it never collides with the intro tape or the final fade.

6.2 ui.tsx — The Typographic Actors

Reveal is the universal entrance: a spring (damping 16, stiffness 130, mass 0.7) drives `translate(dx \cdot (1-p), dy \cdot (1-p))`, a scale from 0.94 to 1, and an opacity clamp of `min(1, 1.6p)`. Nearly every element in the video arrives through it, with per-callsite *dx/dy* offsets choosing the direction of travel. **GlitchText** is the signature effect: every 3 frames it rolls a new random seed; if the roll exceeds 0.72 (and the frame is past 6), it activates for exactly that frame — rendering two ghost copies of the text in `#ff2d6f` and `#00e5ff`, offset in opposite directions, blended with `mixBlendMode screen`, plus (50% of activations) a horizontal slice clipped with `clipPath inset(38% 0 44% 0)` and shoved sideways. Because the probability gate re-rolls deterministically per frame, the glitch strobos organically yet renders identically every time.

Counter rolls numbers with `interpolate(s → e)` under a cubic ease-out and renders with `fontVariantNumeric tabular-nums`, so digit width never changes mid-count — the ACME price ticker and all three fundamental

metrics depend on it. **SceneHeader** assembles a badge pill, a 100 px GlitchText title, a 38 px subtitle, and an underline whose width animates $0 \rightarrow 100\%$ on a second, delayed spring — the animated underline reads as the header underlining itself. **Stamp** is the rubber-stamp slam: a stiff spring (damping 11, stiffness 150) scales from 2.5 down to 1 in a few frames with a rotation, a 6 px border, and layered inner and outer glow, sold by a shockwave and spark burst fired one frame after impact at every call site. **Quote** fades up a 46 px quotation with display-face quote marks and a letterspaced attribution line.

6.3 fx.tsx — The Impact Toolkit

- **Flash** — a full-screen colored flash whose opacity is $\text{peak} \cdot (1 - t/\text{dur})^{2.1}$; the quadratic falloff reads as film burn rather than a blink. Used at every scene entrance and every stamp.
- **Shockwave** — up to N concentric circles staggered 6 frames apart, each living 26 frames: radius eases $16 \rightarrow \text{maxR}$ with cubic ease-out while opacity falls $0.85 \rightarrow 0$. The 'sonic boom' behind every verdict.
- **Streak** — an anamorphic lens flare: a 1900 px horizontal gradient line, blurred, scaling from 0.12 to 3.1 in X while fading, fired exactly when the title's second word lands.
- **Particles** — rising embers: each particle's drift is $(y_0 - f \cdot \text{speed}) \bmod H$ with a ± 30 px sine sway; opacity pulses with an individual phase. Cheap, endless, and it keeps every scene alive.
- **SparkBurst** — radial impact sparks: 22-26 lines emitted from a point with jittered angles, flying outward on an ease-out curve while their length shrinks — the debris of a stamp impact.

6.4 charts.tsx — The Data Visualization Actors

All market charts are drawn as SVG generated from data arrays. **candleMetrics** maps OHLC data into pixel space: it finds the lo/hi of the series, computes candle width from the box, and returns mapX/mapY closures, so the same component renders a 24-candle hero chart or an 8-candle mini chart unchanged. **CandleChart** springs each candle in with a per-index stagger ($\text{start} + i \cdot \text{stagger frames}$): the wick is a 2.2 px line, the body a rounded rect scaled by the spring, colored green when $\text{close} \geq \text{open}$ and red otherwise, each glowing through an SVG drop-shadow filter.

PriceLine is the self-drawing line, and its trick is worth memorizing: the path is built once with a Catmull-Rom spline (`lib.smoothPath`), given `pathLength={1}` and `strokeDasharray={1}`, and then animated purely through `strokeDashoffset` from 1 to $1 - \text{progress}$ — the browser reveals the curve along its own arc length, no clip rects needed. A wider ($\text{width} + 9$) copy at 16% opacity underneath acts as a halo, and a head dot, positioned by the arc-length helper `pointAt(pts, progress)`, rides the tip of the line while it draws, hiding itself in the first and last 3% of the journey. **LevelLine** draws the dashed horizontal support/resistance lines with a right-anchored mono label and a soft glow.

```
const pr = interpolate(f, [start, start + dur], [0, 1],
  {...clamp, easing: Easing.inOut(Easing.quad)});
const d = smoothPath(pts);          // Catmull-Rom -> cubic Bezier path
const head = pointAt(pts, pr);      // arc-length point for the head dot

<path d={d} pathLength={1}
  strokeDasharray={1}
  strokeDashoffset={1 - pr}          // 1 = hidden, 0 = fully drawn
  style={{filter: `drop-shadow(0 0 9px ${color})`}} />
```

Listing 6: The self-drawing line — pathLength, dasharray, dashoffset

6.5 procon.tsx — The Pros and Cons Card

The ProCon card is the workhorse of the two teaching pages. The card itself springs up 54 px into place (damping 16, stiffness 110); its header carries an inline-SVG icon — a rounded check for pros, a cross for cons — and each list item slides in from -38 px on its own spring, staggered 9 frames apart, led by a 9 px diamond bullet (a rotated square) with a color-matched glow. One component, parameterized by kind/title/items/color/at, serves all four cards in the video, which guarantees the fundamental and technical pros-and-cons pages are pixel-symmetric.

7. Scene 1 — Intro: Glitch Title and Candle Skyline

7.1 Purpose and First Impressions

The intro has 4 seconds and one job: make the viewer feel they have plugged into a trading terminal at night. It opens from pure black (a 12-frame black flash fading out) over a green Backdrop with a gold radial halo, 40 twinkling stars, and the GridFloor scrolling at speed 1.7. Two ticker tapes run along the top and bottom edges in opposite directions. At frame 30 — exactly the second beat of the first bar where the drums imply motion — the skyline ignites and the shockwave fires, selling the sense that the market just opened.

7.2 The Candle Skyline

Instead of a city skyline, the horizon is a skyline of **candlesticks**: 26 hand-picked heights (the SKY array, 70-240 px) rise from the bottom edge as spring-animated candle bodies, staggered 2 frames apart starting at frame 30, with a stiff spring (damping 15, stiffness 150, mass 0.5) so they pop up like buildings. Every fifth candle (index 3 mod 5) is red, the rest green, echoing market texture. Each is drawn as three rects — a translucent body (35% opacity), a bright 3 px top cap (90%), and a 14 px wick (70%) — collectively forming a silhouette that reads both as architecture and as a chart.

7.3 Title, Clock Rings, and the Beat Drop

The title block sits at $y = 372$: **MARKET** (white with a mint glow) reveals at frame 7 and **DECODED** (neon green) at frame 29, both at 152 px in Orbitron 900 through GlitchText, so the letters intermittently RGB-split

for the rest of the scene. A letterspaced mono tagline — **FUNDAMENTALS × TECHNICALS**, A 60-SECOND CRASH COURSE — rises at frame 62. Behind the title, two concentric gold rings (radii 234 and 250) breathe with a spring pulse ($\text{scale } 0.9 + 0.1 \cdot p$), prefiguring the beat-pulse language of the VS emblem later. At frame 30 a green Shockwave (max radius 760) and an anamorphic Streak fire across the title line while a 14% green Flash kisses the whole frame — the opening beat drop, synchronized to the soundtrack's first impact.

Element	Frame	Parameters
Black fade-in	0 - 12	Flash, black, peak 0.9 — opens from nothing
MARKET title	7	Reveal dy 0, scale 1.12; GlitchText 152 px, glow #9fffd6
DECODED title	29	Reveal scale 1.18; GlitchText 152 px, green glow
Beat drop FX	30 - 31	Streak (green) at 30; Shockwave r760 at 31; Flash 14% at 29
Candle skyline	$30 + i \cdot 2$	26 candles, spring stiffness 150, mass 0.5
Clock rings	6 / 11	Springs at frames 6 and 11; radii 234 / 250, gold
Tagline	62	Mono 25 px, letter-spacing 0.44em, muted
Ticker tapes	always	Top $y=0 \rightarrow \text{direction } +1$; bottom $y=1024 \rightarrow \text{direction } -1$

Table 7: Scene 1 element timing (scene-local frames)

8. Scene 2 — The Hook: Self-Drawing Charts and a Live Ticker

8.1 The Two Questions

Scene 2 dramatizes the thesis in a single crossfade. At frame 4 a kicker line — **EVERY INVESTOR ASKS** — fades down into place. The first question, **WHAT DO YOU BUY?** (white, 92 px GlitchText), holds until frame 74, then crossfades over frames 74-94 into **WHEN DO YOU BUY?** (cyan) while scaling $0.94 \rightarrow 1$. The pair of questions is the entire video in miniature: fundamental analysis answers the what, technical analysis answers the when — and the viewer has just been taught that before a single term was defined.

8.2 A Market That Draws Itself

Below the questions, a 24-candle OHLC series (hand-authored to trend upward with two realistic pullbacks) draws itself inside a 1580×410 chart box at (170, 396). Candles begin springing in at frame 16 with a 3-frame stagger; at frame 58 a cyan PriceLine starts tracing the close prices over 70 frames, its glowing head dot riding the tip. Beneath the chart, a terminal chip labeled **ACME GROWTH FUND** hosts a live Counter rolling from \$1,024.50 to \$1,248.90 across frames 30-118 next to a +21.9% pill in a green-tinted border — the chart, the number, and the percentage all agree, which is what makes the fake market feel real.

8.3 Planting the Rivalry

At frames 132 and 140 the two schools slide in from opposite sides as bordered pills: **01 · FUNDAMENTAL** — **the WHAT** in green from the left, **02 · TECHNICAL** — **the WHEN** in cyan from the right, with a gold VS dropping between them at frame 150 as a double gold shockwave ripples across the bottom of the frame. This is the hand-off to the next two scenes: the pills introduced here are the exact colors and labels the teaching scenes inherit. A white 22% entrance flash at frame 0 completes the scene cut from the intro's green world into the cyan world of the hook.

Element	Frame	Detail
Kicker EVERY INVESTOR ASKS	4	Reveal dy -18, mono 28 px, spacing 0.5em
Question 1: WHAT DO YOU BUY?	0 - 84	White 92 px; opacity 1 → 0 over 74-84
Question 2: WHEN DO YOU BUY?	84 - end	Cyan 92 px; opacity 0 → 1 over 84-94
Candle chart	16 + i:3	24 candles, box 1580 × 410 at (170, 396)
Close-price line	58 - 128	Cyan, width 5, head dot on
Price counter	30 - 118	\$1,024.50 → \$1,248.90, cubic ease-out, 2 decimals
School pills + VS	132 / 140 / 150	Green +1 / cyan -1 slides; gold VS; 2-ring shockwave

Table 8: Scene 2 key timings (scene-local frames)

9. Scene 3 — Fundamental Analysis (School 01)

9.1 Page 1: The Company Report Card

Scene 3 teaches the first school in green. The SceneHeader slams in at frame 4 — badge **SCHOOL 01**, title **FUNDAMENTAL ANALYSIS**, subtitle **Value the business** — then **buy it on sale** — followed by the scene's centerpiece: a 980 × 390 glass card for **ACME CORP**, framed by a 1.5 px green border and a 60 px green glow. Its left half is a five-year revenue bar chart (values 0.30, 0.44, 0.38, 0.60, 0.82, scaled to 158 px), each bar springing up with a 7-frame stagger from frame 40, filled with a vertical green gradient and labeled Y1-Y5. Its right half rolls three Counter metrics with staggered windows: **REV GROWTH +18.2%** (frames 46-104), **NET MARGIN 21.4%**, and **DEBT / EQUITY 0.4×** — the three numbers a fundamental investor actually reads first.

To the right of the card, the valuation gap is staged as two chips: **PRICE \$95.00** (white, at frame 62) above **VALUE \$120.00** (gold, glowing, at frame 80). At frame 98 the **UNDERVALUED** stamp slams across them at -7°, firing a double gold shockwave and a 24-spark burst one frame later — the loudest moment of the scene, placed on a musical impact. A caption at frame 126 completes the teaching: It reads the company's report card — earnings, debt, growth, competition.

The math on screen is honest: a \$95 price against a \$120 intrinsic value means the buyer pays $95/120 \approx 0.79$ of fair value — a 21% discount (and 26.3% upside to fair value). The stamp, the shockwave, and the caption BUYING AT A 21% DISCOUNT all state the same number.

9.2 The Page Turn

Between frames 156 and 172 the entire first page (header, card, chips, stamp) fades to 0 while translating up 34 px — a page exit that feels like sliding a sheet off a lightbox. The second page is already beneath it: a small green kicker (SCHOOL 01 · FUNDAMENTAL ANALYSIS) at top-left, the gold headline WEIGHING THE APPROACH at frame 178, and at frames 184 and 196 the two ProCon cards slide up, followed by the Warren Buffett quotation — Price is what you pay. Value is what you get. — at frame 300 with a soft gold flash marking the page turn at 172.

PROS (green card, frame 184)	CONS (red card, frame 196)
Reflects the real business	Slow and research-heavy
Built for long-term wealth	Forecasts can be wrong
Fewer false signals	Blind to market mood
Less day-to-day noise	Irrational markets persist

Table 9: The pros and cons as presented (Scene 3, page 2)

Note the deliberate balance: four claims each, every claim one concrete sentence, no school getting the last word in its own scene — the comparison is deferred to Scene 5 so both pages stay fair. The same structure (identical positions, identical timing, only hue and content swapped) is reused verbatim by Scene 4, which is why the viewer experiences the two schools as equal contenders.

10. Scene 4 — Technical Analysis (School 02)

10.1 Page 1: Anatomy of a Chart

Scene 4 is Scene 3's mirror in cyan: same header rhythm (badge SCHOOL 02, title TECHNICAL ANALYSIS, subtitle Read the chart — let price tell the story), same page-turn at frames 156-172, same pros-and-cons stage. But its first page is a live chart-reading lesson built from five coordinated instruments inside a 1140×376 panel. Two dashed **LevelLines** set the arena — RESISTANCE in red at $y = 434$, SUPPORT in green at $y = 728$, each with a glowing mono label. Twenty candles draw in from frame 34 (4-frame stagger), and two hand-tuned moving averages are traced on top: a **fast MA** in gold and a **slow MA** in violet, drawn as PriceLines (width 4.5, no head dot) over 52 frames from frame 44. The FAST array was authored so that it crosses above SLOW at index 10 — at that exact point a GOLDEN CROSS stamp slams with a double gold shockwave (frame 96), naming the pattern at the moment it happens.

10.2 The Bouncing Price Ball

The most playful element teaches the most practical idea: price keeps testing the levels. From frame 26, a 26 px white ball with a cyan glow bounces vertically between resistance and support on a triangle wave — $bt = (f - 26) \cdot 0.058$, $y = 434 + |(bt \bmod 2) - 1| \cdot (728 - 434)$ — and at each contact it squashes to 0.78 vertical scale (and stretches horizontally) for the frames where the wave is within 7% of an extreme. That squash-and-stretch, lifted straight from animation principles, is what turns a math formula into something that feels like it has mass.

```
const bt = Math.max(0, (f - 26) * 0.058);
const tri = Math.abs((bt % 2) - 1); // 0..1..0 ping-pong
const bally = 434 + tri * (728 - 434); // resistance..support
const squash = tri > 0.93 || tri < 0.07 ? 0.78 : 1;
// render: scaleY(squash) scaleX(2 - squash) for volume conservation
```

Listing 7: The bouncing ball — triangle wave plus contact squash

10.3 RSI and Volume Panels

To the right, two instrument panels complete the terminal. The **RSI 14 · MOMENTUM** panel (500 × 260) draws dashed red and green reference lines at the 70 and 30 bands, then a cyan oscillator path — 40 samples of $50 + 34 \cdot \sin(0.42i + 1.1) + 6 \cdot \sin(0.9i)$, clamped to 2-98 — that draws itself on with a smoothstep ease across frames 58-130, visibly surging above 70 and diving under 30. Below it, the **VOLUME** panel (500 × 130) springs up 16 bars (3-frame stagger from frame 70) whose heights come from seeded random $(0.2 + \text{rnd} \cdot 0.8) \times 74$ px, colored green or red by another seeded roll — deterministic, so the same bars appear in every render and every QA still. A caption at frame 126 reads: It reads the tape — price action, volume, patterns, momentum.

PROS (green card, frame 184)	CONS (red card, frame 196)
Fast, visual, real-time	False breakouts and whipsaws
Works on any timeframe	Ignores the business
Clear entries and exits	Crowded and self-fulfilling
Defines risk with stop-losses	Invites overtrading

Table 10: The pros and cons as presented (Scene 4, page 2)

The page closes with the trader's adage — The trend is your friend. — in cyan, mirroring Scene 3's Buffett quote. Scene 4's entrance flash is cyan at 26%, and its page turn fires the same gold 10% flash as Scene 3, keeping the mirrored grammar intact down to the flash colors.

11. Scene 5 — Head-to-Head: The Versus Scene

11.1 A Stage Split in Two

Scene 5 is the debate, and its gold accent signals that it belongs to neither school. The backdrop gets two opposing tint washes — a green gradient bleeding in from the left edge, a cyan one from the right — so each half of the screen is quietly owned by a school. Down the center runs a hand-built **lightning divider**: a polyline of 26 segments that zig-zags around $x = 960$ with $\pm(7 + \text{rnd} \cdot 9)$ px of horizontal jitter per vertex, drawn on top-to-bottom by a spring (strokeDashoffset trick) and then flickering forever — its opacity re-rolls every 4 frames between 0.68 and 1.0, layered with two drop-shadows of different radii for a electric-blue bloom. The column headers FUNDAMENTAL (green) and TECHNICAL (cyan) drop in at frames 8 and 13.

11.2 The VS Emblem and the Four Rounds

At frame 15 a 150 px gold **VS** emblem scales in ($1.5 \rightarrow 1$) at the center of the frame — and it never sits still: its scale is $1 + 0.05 \cdot \text{beat}(f)$, so it visibly pumps on every kick drum, and its text shadow is chromatic (a red ghost at -6 px, a cyan ghost at +6 px), pre-echoing the GlitchText language. The comparison itself is four rows that arrive like rounds of a fight, each 45 frames after the previous (frames 38, 83, 128, 173), at 142 px vertical spacing: a faint letterspaced label on both sides, then the fundamental answer sliding from the left (dx -110) and the technical answer from the right (dx +110), each in a bordered chip tinted with its school color.

Round	Fundamental says	Technical says
THE QUESTION	WHAT to buy?	WHEN to buy?
TIME HORIZON	Years → decades	Minutes → months
THE TOOLKIT	Financials · economy · moat	Charts · patterns · momentum
THE MINDSET	Own great businesses	Ride the wave, cut fast

Table 11: The four comparison rows (the debate in full)

11.3 The Verdict

At frame 252 the four rows compress — the whole container scales to 0.88 and dims to 0.3 opacity over 16 frames — clearing the stage while never disappearing. Into that space at frame 272 the verdict stamps: **WHY NOT BOTH?** at 78 px in gold, -5° rotation, followed one frame later by the largest shockwave in the video (three rings, max radius 900) and a 26-spark burst. The resolution line — The best investors borrow from both books. — rises at frame 302, and a gold 20% flash sells the impact. This scene is the pivot of the whole argument: it gives the fight a winner that is not either fighter.

12. Scene 6 — Outro: The Synthesis and the End Card

12.1 Merging the Two Schools

The outro shifts to violet — deliberately outside both schools' hues. Its first movement is a visual thesis: the fundamental school's mini candle chart (8 candles) springs in from the left (translateX from -220 px on a spring at frame 8) while the technical school's cyan price line slides in from the right (+220 px at frame 14); at frames 98-116 both fade out — they have merged. In their place a single **golden RISE line** (seven hand-placed points climbing from (620, 660) to (1318, 338)) draws itself from frame 40 to 96 with its head dot, and when it reaches the summit an arrowhead polygon fades in at frames 97-106, crowned by a gold shockwave (radius 420) and a 24-spark burst. Two lenses, one mission — literally one line drawn from two.

12.2 The Lesson, in Three Lines

The synthesis is spelled out as a three-beat stack: the glitch headline **TWO LENSES. ONE MISSION.** (86 px, gold glow) at frame 104; then two 46 px lines — **FUNDAMENTALS** pick the **WHAT** (the word **FUNDAMENTALS** in green) at frame 154, **TECHNICALS** time the **WHEN** (**TECHNICALS** in cyan) at frame 172; and finally **SMART MONEY USES BOTH** in letterspaced gold mono at frame 224. Each line arrives on its own Reveal so the lesson lands in rhythm rather than as a paragraph.

12.3 The End Card and the Long Fade

The exit is choreographed in stages so the screen never clutters: the golden line fades away over frames 286-316, the text block over 300-330 (staggered, so the eye drains downward), and the end card fades in over 316-340 — **MARKET DECODED** at 88 px (**DECODED** in green) with a green shockwave at 334, and beneath it the compliance line **EDUCATIONAL ONLY · NOT FINANCIAL ADVICE** in mono. After frame 404 a 16 px green heartbeat dot blinks on and off in 9-frame cycles at the exact center of the frame — the terminal's cursor, still alive. From frames 392 to 416 a black overlay fades to full, and the video ends on true black, one beat before the music's final tail resolves.

Movement	Frames	Detail
School charts fly in	8 - 14	Mini candles from -220 px; cyan line from +220 px
Merge	98 - 116	Both charts fade out together
Golden RISE line	40 - 96	Self-drawing, head dot, arrowhead at 97-106
Headline + lesson	104 - 224	Glitch headline; WHAT / WHEN lines; SMART MONEY
Staged exits	286 - 330	Line fades 286-316; text fades 300-330
End card	316 - 344	Title + disclaimer; green shockwave at 334
Heartbeat + blackout	404 / 392 - 416	9-frame blink; fade to #000

Table 12: Scene 6 master timeline (scene-local frames)

13. Quality Assurance: The Frame Review Loop

13.1 Why Still Frames Are Enough

Remotion's determinism turns QA into a tractable problem: every frame is a pure function of its frame number, so any layout bug lives at a specific, reproducible frame. Instead of scrubbing 60 seconds of video, the loop renders individual frames with remotion still and inspects them as images. Fourteen stills were rendered for the first pass — frames 70, 240, 430, 560, 760, 830, 940, 1160, 1320, 1560, and 1740, plus targeted checks at the 7 s and 29 s marks — chosen to sample every scene's most crowded moments (entrances, stamps, page turns, the versus rows).

```
npx remotion still src/index.ts StockMarketDecoded \
  out/qa/f430.png --frame=430
# repeat per frame; then visually inspect each PNG
```

Listing 8: The QA command — reproducible still frames

13.2 Six Bugs Found and Fixed

The first pass caught six real defects — every one of them a layout collision or a missing exit that only reveals itself at a specific frame. The table records each bug and its fix exactly as applied.

Defect	Where	Fix applied
HUD clock showed raw seconds (1:05 impossible in a 60 s video)	HUD	Reformatted to m:ss with padStart; verified 0:xx throughout
Progress HUD collided with the intro's bottom ticker tape	S1 / HUD	HUD now fades in at frames 128-152, after the intro ends
UNDERVALUED stamp overflowed the card's right edge	S3	Stamp anchor moved to (1626, 632); rotation kept at -7°
100 px scene header overlapped the report card / chart panel tops	S3 / S4	Header block tightened; content y-coordinates pushed down
VS emblem collided with row-3 chips at the frame center	S5	Rows compress to 0.88 scale and dim to 0.3 before the stamp; VS inherits the dim
Scene 6 text never cleared before the end card arrived	S6	Added staged fades: line 286-316, text 300-330

Table 13: The QA ledger — defect, frame, fix

13.3 The Second Pass

After the fixes, four fresh stills (v2 frames 430, 760, 1160, 1740 — the exact frames where four of the bugs lived) were rendered and confirmed clean: the stamp sits inside the card, the headers breathe, the VS emblem owns the frame after the rows yield, and the end card fades in over an empty stage. Only then was the full render attempted. This is the quiet advantage of programmatic video: QA is cheap, precise, and repeatable, and a fix can be verified against the exact frame that exposed the bug.

14. Rendering and Delivery

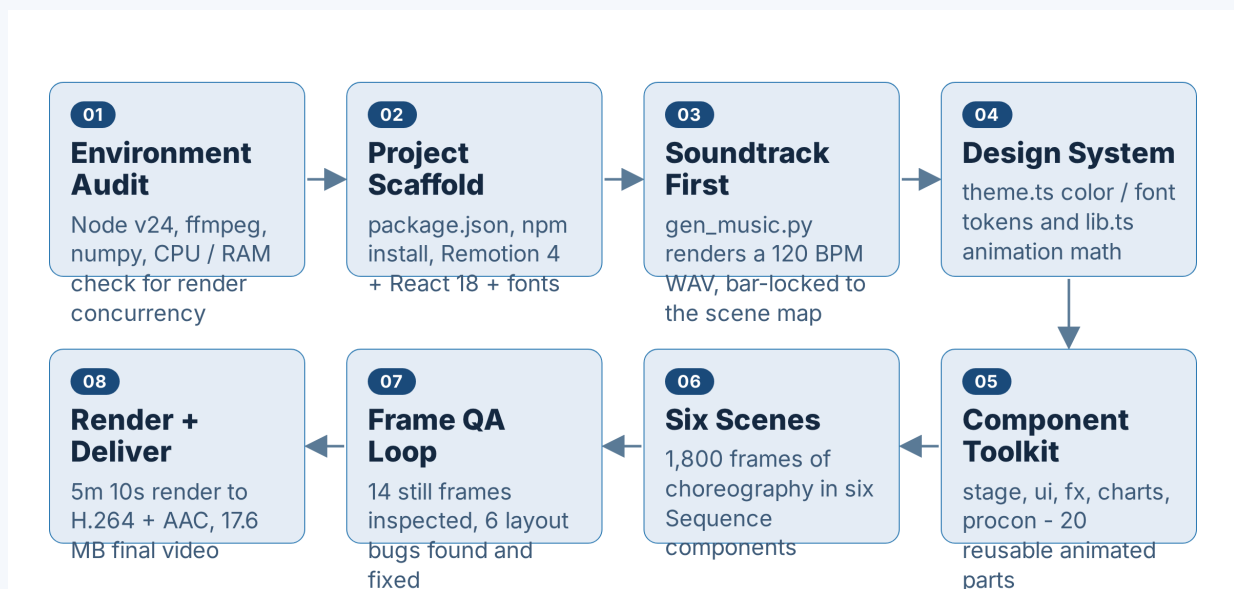


Figure 2: The full production pipeline, from environment audit to delivery

14.1 Benchmark Before the Big Render

Before committing to the full 1,800-frame render, a 120-frame benchmark was run — roughly two bars of the hook, dense with springs and glitch frames — and completed in 38.7 seconds at concurrency 2. Linear extrapolation predicted about 9.7 minutes for the full video; in practice the full render finished in **5 minutes 10 seconds**, because the benchmark's cold browser start and font compilation are amortized across a longer run. The lesson generalizes: benchmark a slice, extrapolate, then let the real render surprise you on the good side.

120 f benchmark slice	38.7 s benchmark time	5m 10s full render	2 browser tabs
--------------------------	--------------------------	-----------------------	-------------------

14.2 The Render Command, Dissected

```

npx remotion render src/index.ts StockMarketDecoded \
  out/stock-market-decoded.mp4
# entry point      -> src/index.ts (registerRoot)
# composition id   -> StockMarketDecoded (1800 f, 30 fps, 1920x1080)
# output          -> H.264 MP4; <Audio> muxed to AAC automatically

```

Listing 9: The one command that produces the deliverable

The command names the bundle entry point and the composition ID, and Remotion does the rest: it bundles `src` with `esbuild`, boots 2 headless Chromium tabs (the concurrency from Chapter 2), screenshots frames 0-1799 as JPEGs, streams them into its bundled `ffmpeg` encoder as H.264, and muxes the `music.wav` referenced by the Audio tag as AAC — one command, one deterministic file. `OverwriteOutput` made iteration safe: each QA fix in Chapter 13 was followed by simply re-running this command.

14.3 Delivery

The finished MP4 (17.6 MB) was copied to the download directory as `stock-market-decoded.mp4` alongside a small README pointing back to the project source. The project itself remains fully re-renderable: the source tree, the music script, and this document together form a complete, self-contained production kit.

15. Complete File Inventory and Reproduction Guide

15.1 Every File and Its Role

File	Role
<code>remotion-video/package.json</code>	Dependency manifest; <code>npm run render</code> shortcut
<code>remotion-video/remotion.config.ts</code>	Renderer tuning: JPEG frames, concurrency 2, ANGLE, overwrite
<code>remotion-video/tsconfig.json</code>	Strict TypeScript config; type-check only (noEmit)
<code>remotion-video/src/index.ts</code>	Entry point; registers the Remotion root
<code>remotion-video/src/Root.tsx</code>	Declares the <code>StockMarketDecoded</code> composition (1800f, 30fps, 1080p)
<code>remotion-video/src/StockMarketVideo.tsx</code>	Timeline host: Audio, six Sequences, HUD, ScreenFX
<code>remotion-video/src/theme.ts</code>	COLORS, FONTS, SCENES — the single source of truth
<code>remotion-video/src/lib.ts</code>	<code>clamp</code> , <code>beat()</code> , <code>smoothPath</code> (Catmull-Rom), <code>pointAt</code> , <code>rnd</code> , <code>easing</code>
<code>remotion-video/src/components/stage.tsx</code>	Backdrop, GridFloor, StarField, TickerTape, ScreenFX, ProgressHUD
<code>remotion-video/src/components/ui.tsx</code>	Reveal, GlitchText, Counter, SceneHeader, Stamp, Quote

File	Role
remotion-video/src/components/fx.tsx	Flash, Shockwave, Streak, Particles, SparkBurst
remotion-video/src/components/charts.tsx	CandleChart, PriceLine, LevelLine, candle metrics helpers
remotion-video/src/components/procon.tsx	The pros/cons card with icon and staggered items
remotion-video/src/scenes/S1Intro.tsx	Glitch title, candle skyline, clock rings, double ticker tape
remotion-video/src/scenes/S2Hook.tsx	Two questions crossfade, self-drawing chart, price counter
remotion-video/src/scenes/S3Fundamental.tsx	ACME report card, valuation gap, UNDERVALUED stamp, pros/cons
remotion-video/src/scenes/S4Technical.tsx	Support/resistance, golden cross, bouncing ball, RSI, volume
remotion-video/src/scenes/S5Versus.tsx	Lightning divider, VS emblem, four rounds, WHY NOT BOTH stamp
remotion-video/src/scenes/S6Outro.tsx	Merging charts, golden RISE line, lesson, end card, blackout
remotion-video/public/music.wav	The synthesized 60 s soundtrack (AAC source at render time)
scripts/gen_music.py	NumPy synth — regenerates music.wav from the scene map
download/stock-market-decoded.mp4	The delivered video (17.6 MB)

Table 14: The complete project inventory

15.2 Reproducing the Video from Scratch

1. Install Node.js 18 or newer and confirm ffmpeg is available on PATH.
2. Create the project folder and copy in package.json (Listing 1); run npm install.
3. Regenerate the soundtrack: python3 scripts/gen_music.py writes public/music.wav (or keep the existing WAV).
4. Start the live preview studio with npx remotion studio — every scene can be scrubbed frame by frame in the browser.
5. Render the master: npm run render, which executes the command dissected in Chapter 14.
6. QA any suspicious moment with npx remotion still --frame=N, exactly as described in Chapter 13.

15.3 Where to Take It Next

- **Re-theme in one file** — every color and font lives in theme.ts; changing COLORS.green or FONTS.display re-styles all six scenes.
- **Re-time the whole video** — the SCENES map shifts scene boundaries, HUD chapter ticks, and (via gen_music.py) the musical arrangement together.

- **Swap the market data** — scenes take plain OHLC arrays; feeding real historical candles turns the video into a market recap generator.
- **Parameterize the composition** — Remotion comps accept props; exposing company name, prices, and metrics would make this a template for any stock.
- **Scale the render** — on a bigger machine raise `setConcurrency`, or render on Remotion's cloud lambda renderer for near-real-time output.

15.4 Performance Notes for Faster Renders

Rendering is the slowest loop in the pipeline, and the 2-core, 4 GB machine used for this build made its management explicit. The full render took 5 minutes 10 seconds, but every minute of that was earned through the small disciplines recorded below - most of them configured once in `remotion.config.ts` and then forgotten. Anyone adapting this project to new content will hit the same five bottlenecks in the same order.

Bottleneck	Remedy used in this build
Slow full renders	Benchmark a 120-frame slice first (38.7 s here) and extrapolate before committing to the full run
Memory pressure from browser tabs	Concurrency clamped to physical cores (2 here); JPEG frame format keeps each tab light
Long QA cycles	remotion still on exact suspect frames - seconds per check instead of minutes per re-render
Slow authoring previews	Scrub a single Sequence in the studio while tuning one scene; the other five stay unmounted
Scaling out	Raise <code>setConcurrency</code> on bigger machines, or move to Remotion Lambda for parallel cloud rendering

Table 15: Render bottlenecks and the remedies used in this build

The project demonstrates a workflow, not just a video: lock a beat grid, generate the soundtrack from the same map, build a small deterministic component toolkit, compose scenes from those parts, QA exact frames, and render with one command. Any future motion-graphic idea — crypto, sports, elections, product launches — can ride the same pipeline with new data and a new palette.